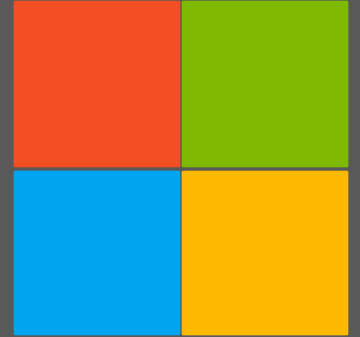


UNIVERSITY OF WASHINGTON | FOSTER SCHOOL OF BUSINESS
MSBA CAPSTONE × MICROSOFT



CLOUD SECURITY · RISK ANALYTICS · MAY 2026

Cloud Risk Assessment Model

A dual-model framework for scoring vendor cybersecurity risk

TEAM

Arhum · Hansen · Kimmich · Nelson · Shelby · Shivani · Ximena

SPONSOR

Microsoft Corporation

FACULTY ADVISOR

UW Foster School of Business

Project framing

Objective

Build an end-to-end cloud risk assessment pipeline that ingests live data from the Azure MySQL CloudRisk database, engineers predictive features, and produces a per-service risk score and anomaly flag for use in a Power BI dashboard.

Approach

Compare five machine-learning models on a stratified train/test split, then run two of them in parallel in production: a primary supervised scorer for known risk patterns, and an unsupervised anomaly detector for novel patterns the historical label does not cover.

**33,181****Vulnerability records ingested***Source: vulnerabilities table***5****Machine-learning models benchmarked***Section 5 of technical summary***2****Models running in parallel production***XGBoost + Isolation Forest*

Agenda

01

Introduction

Project framing and approach

02

Data & Architecture

Four Azure MySQL tables and pipeline design

03

Feature Engineering

Six categories of service-level signals

04

Model Development

Five models, configurable risk weights

05

Dashboard & Drill-Down

Power BI integration and analyst workflow

06

Business Relevance

Operational value, limitations, next steps

Four tables, one Azure MySQL pipeline

All four tables are joined at the service level on `service_id`. The `services` table was found to be empty in the live database and is reconstructed by taking distinct `service_id` / `service_name` pairs from the `vulnerabilities` table.

PRIMARY

33,181 rows

vulnerabilities

CVE / CWE records with CVSS v3, EPSS, KEV flag, ecosystem

`cve_id` · `cwe_id` · `cvss_v3_score` · `epss_score` · `is_kev`

INCIDENTS

263 rows

service_incidents

Historical incident records — severity, status, downtime

`incident_id` · `severity` · `status` · `duration_minutes`

REPO

24 rows

repo_health

Repository-level development and security health signals

`commit_count_90d` · `security_issues_open` · `days_since_last_release`

METADATA

Reconstructed

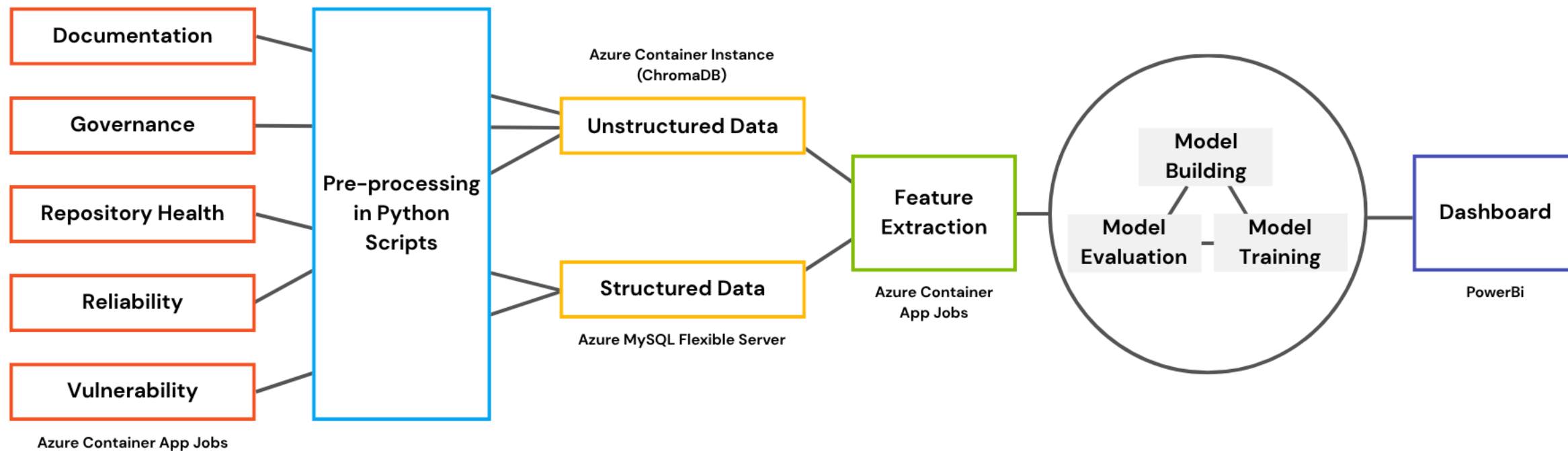
services

Empty in production — rebuilt from vulnerabilities table

`service_id` · `service_name`

Pipeline architecture

End-to-end flow from Azure MySQL ingestion through feature engineering, model training, and Power BI consumption. Full architecture diagram below.



Data sources — vulnerabilities

PRIMARY

vulnerabilities

33,181 rows

Purpose

Source of all per-CVE risk signals used by the model — severity, exploitability, and attack class.

How it's used

Aggregated per service into 7 numeric features (total_vulns, open_vulns, mean_cvss, max_cvss, exploit_count, mean_epss) plus 20 one-hot CWE flags.

KEY FIELDS

cve_id	Unique CVE identifier
cwe_id	Common Weakness Enumeration (attack class)
severity	Categorical: critical / high / medium / low
cvss_v3_score	0–10 technical severity score
epss_score	0–1 exploitation probability (FIRST EPSS)
is_kev	Binary: on CISA Known Exploited Vulnerabilities list
service_name	Join key to other three tables
ecosystem	Package ecosystem (npm, pypi, maven, etc.)

Data sources — service_incidents

INCIDENTS

service_incidents

8 services with incident history

Purpose

Historical evidence of real-world security or availability failures per service.

How it's used

Aggregated into incident_count, avg_severity, recurrence_rate (unresolved share), and total_downtime. Sparse coverage — only 8 services have incident history in current dataset.

KEY FIELDS

incident_id	Unique incident identifier
service_name	Join key to other three tables
severity	critical / high / medium / low (mapped to 1–4)
status	resolved / open / investigating
duration_minutes	Cumulative downtime
started_at	Incident start timestamp
resolved_at	Incident resolution timestamp

Data sources — repo_health

REPO

repo_health

24 repository snapshots

Purpose

Code-base maintenance signals indicating how actively a service is being developed and patched.

How it's used

Derived feature $\text{patch_debt} = \text{days_since_last_release} \times \text{security_issues_open}$ combines staleness and unresolved security work into a single signal.

KEY FIELDS

service_name	Join key
repo_full_name	GitHub-style owner/repo
commit_count_90d	Commit activity in last 90 days
contributor_count_90d	Distinct contributors in last 90 days
open_issues	Open GitHub issues
security_issues_open	Open security-labelled issues
mean_days_close_security	Average days to close security issues
days_since_last_release	Staleness of most recent release
has_security_policy	Binary — repo has SECURITY.md

Data sources — services

METADATA

services

2,865 unique services

Purpose

Service-level descriptive metadata used for grouping and stratification.

How it's used

Reconstructed at runtime from distinct service_name pairs in vulnerabilities when source table is sparse. Categorical fields label-encoded for model input.

KEY FIELDS

service_id	Numeric service identifier
service_name	Primary join key across all four tables
ecosystem	Technology ecosystem (npm, pip, maven, SaaS, etc.)
vendor	Vendor / publisher name
is_oss	Binary — open-source service
is_saas	Binary — SaaS service

Six categories of service-level signals

Features are aggregated to the service level and grouped into six categories. Each category carries a configurable weight that amplifies its features during model training.

Vulnerability Severity

How dangerous are the CVEs?

`mean_cvss · max_cvss · mean_epss`

Vulnerability Volume

How many CVEs are there?

`open_vulns · total_vulns · vuln_density`

Active Exploitation

Any actively exploited?

`exploit_count · exploit_ratio · epss_risk`

Incident History

Real-world failures?

`avg_severity · recurrence_rate · total_downtime`

Repository Health

Is the codebase maintained?

`patch_debt · security_issues_open ·
commit_count_90d`

CWE Attack Types

What attack patterns?

20 one-hot CWE flags (XSS, SQLi, etc.)

→ *high_risk* label triggers if ANY: `mean_cvss ≥ 7.0 · exploit_count ≥ 1 · mean_epss ≥ 0.1 · incident_count ≥ 3`

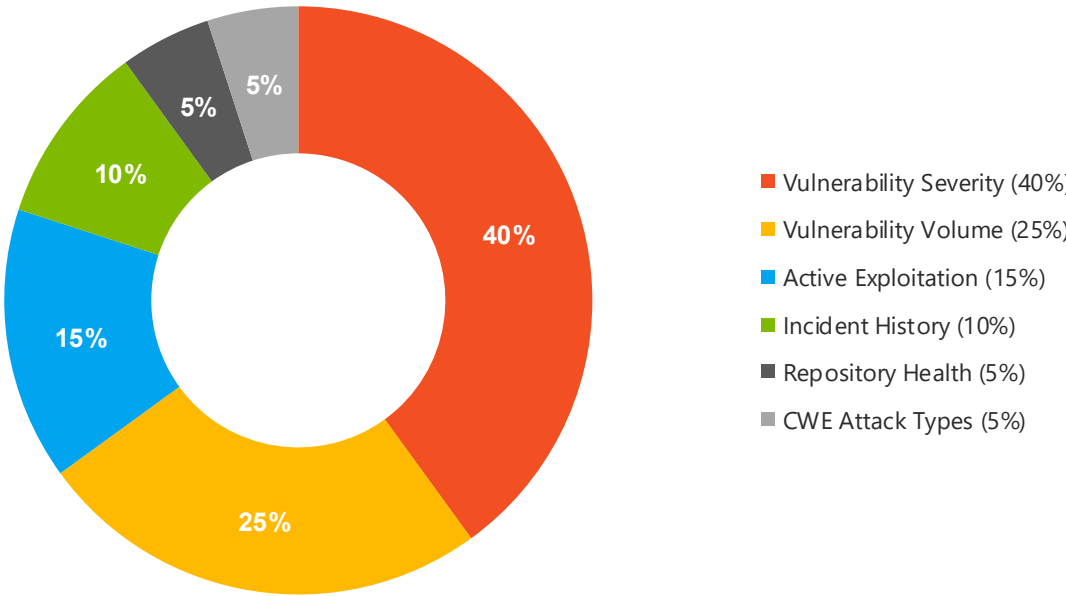
Configurable threshold-based risk label

Unlike a percentile-based label, the model uses explicit thresholds defined by the security team. A service is labelled `high_risk = 1` if it crosses ANY of four conditions. All thresholds and the six category weights are set in Section 0 of the notebook.

HIGH-RISK CONDITIONS (any one triggers label = 1)

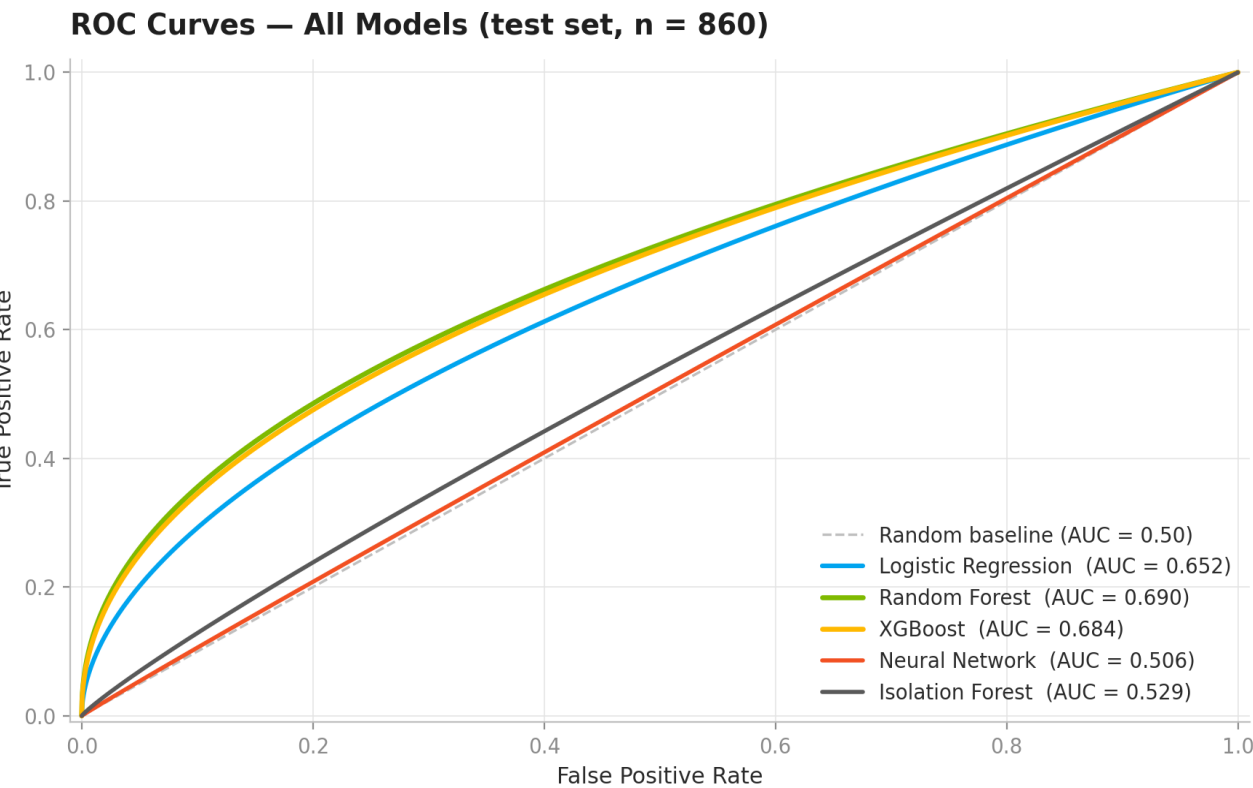
mean_cvss	≥ 7.0
CVSS "High" severity boundary	
exploit_count	≥ 1
Any KEV-listed CVE present	
mean_epss	≥ 0.10
≥10% exploitation probability	
incident_count	≥ 3
Repeated historical failures	

CATEGORY WEIGHTS (default — configurable)



Five models benchmarked on a held-out test set

Models trained on the weighted feature matrix with a stratified train/test split (2,005 train / 860 test). Primary selection metric is 5-fold cross-validation AUC for stability.

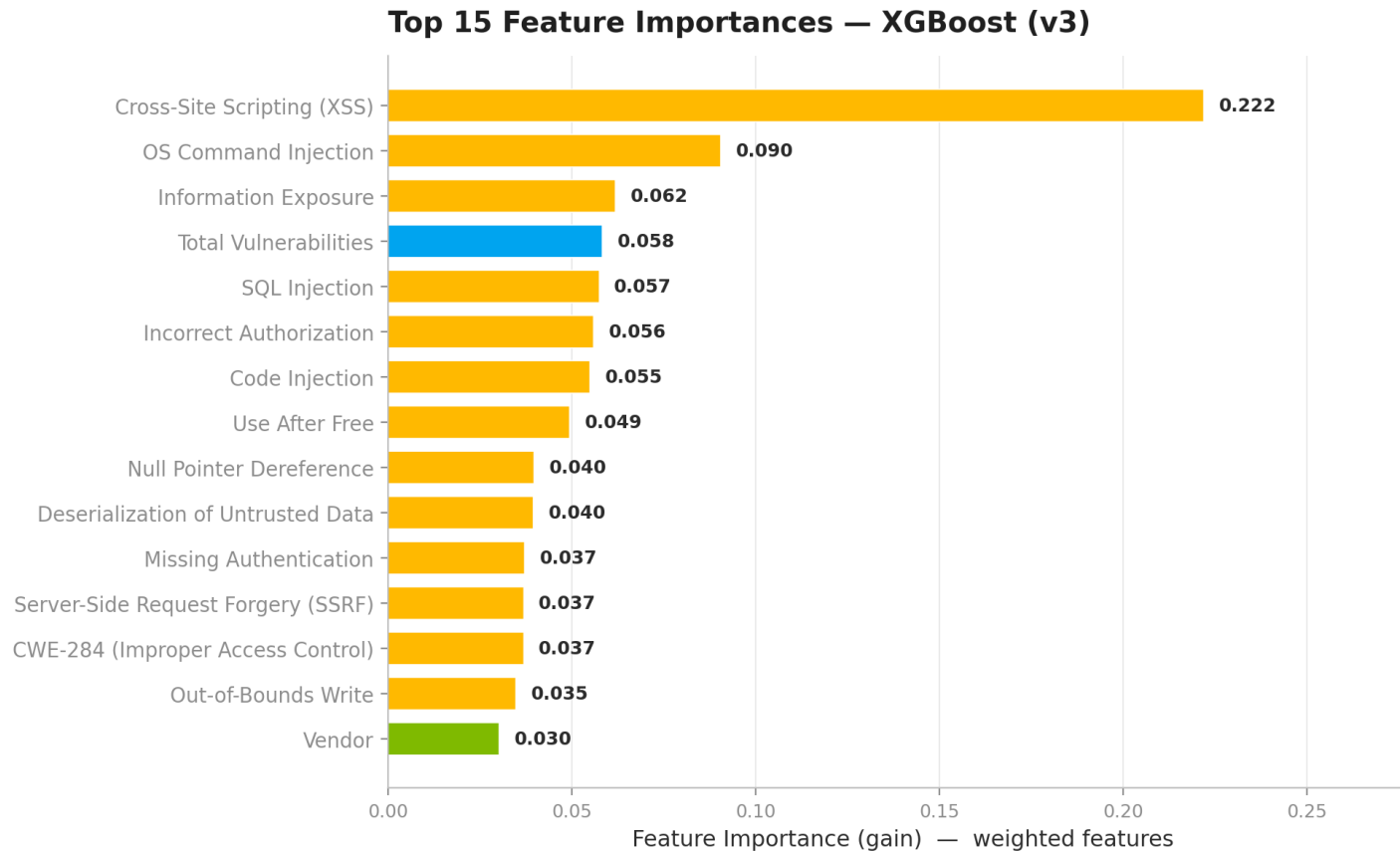


Model	Test AUC	CV AUC	Precision	Recall	F1
Logistic Regression	0.652	0.628	0.782	0.384	0.515
Random Forest	0.690	0.716	0.703	0.876	0.780
XGBoost	0.684	0.710	0.697	0.836	0.760
Neural Network	0.506	0.556	0.631	0.985	0.770
Isolation Forest	0.529	—	0.545	0.011	0.022

Note: RF and XGBoost tie within 1% on CV AUC. XGBoost is selected as production primary for feature-importance interpretability and SHAP compatibility.

What drives the risk score

XGBoost feature importances on the weighted feature set. Note: high-weight categories tend to rank higher because their features were amplified during training.



1 XSS dominates

Cross-Site Scripting alone accounts for ~22% of total gain — the dataset is heavy in web-facing services.

2 Injection family stacks

OS Command, SQL, and Code Injection together rank in the top 7 — classic web vulnerability surface.

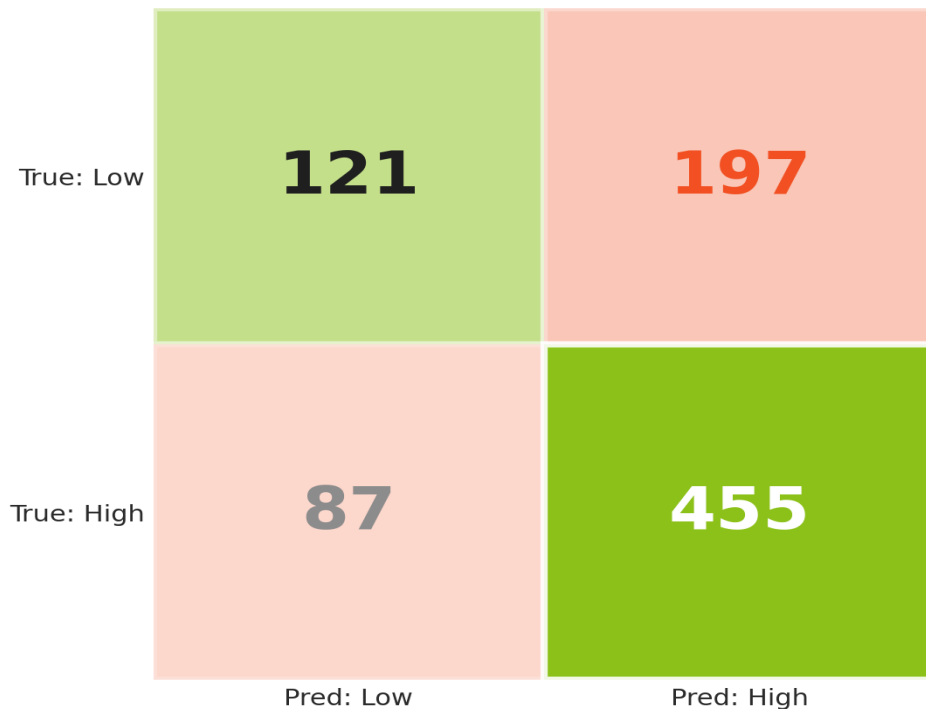
3 Volume still matters

Total Vulnerabilities is the only non-CWE numeric feature in the top 5 — sheer count is a real signal.

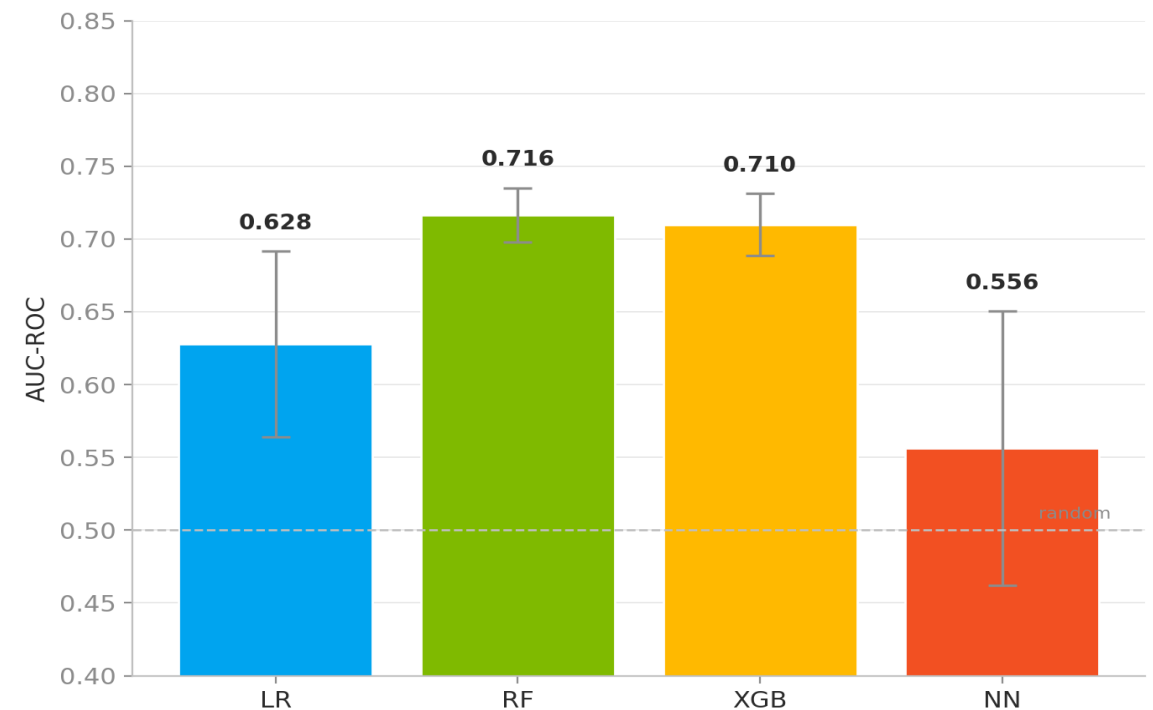
Robustness — confusion matrix and cross-validation

Two diagnostics confirm model behaviour on the model dataset. XGBoost recalls 84% of high-risk services correctly at the cost of false positives, and 5-fold CV confirms RF and XGBoost as the stable leaders.

Confusion matrix — XGBoost (test set, n = 860)



5-fold cross-validation AUC $\pm$ std



Production design — two models in parallel

Production runs two models in parallel: a supervised primary scorer for known risk patterns, and an unsupervised anomaly detector for novel patterns the historical label does not cover.

XGBoost

Primary risk scorer

- Highest CV AUC of supervised models (0.710 ± 0.021)
- Calibrated 0–1 probability per service via `predict_proba`
- Feature importances available out-of-the-box, SHAP-compatible
- Directly consumable by Power BI as `xgb_risk_score`

BEST CV AUC · INTERPRETABLE

Isolation Forest

Parallel anomaly detector

- Unsupervised — no incident label required
- Outputs `anomaly_score` (0–1) and binary `anomaly_flag`
- Flags services whose attribute combination is statistically unusual
- Catches novel risk patterns the threshold label cannot define

NOVEL · ANOMALY

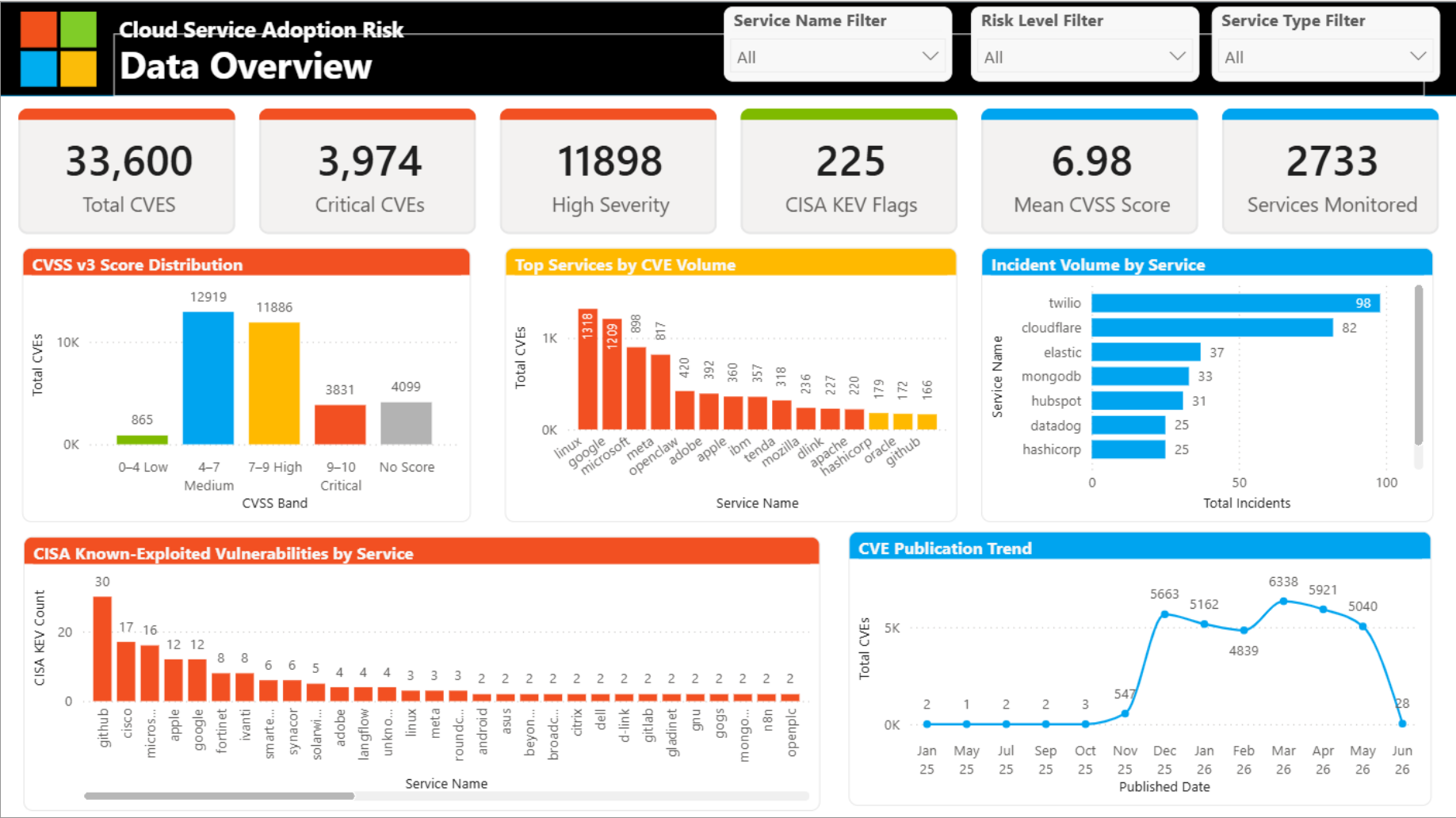
Risk-score band →

 Low · 0.00 – 0.40

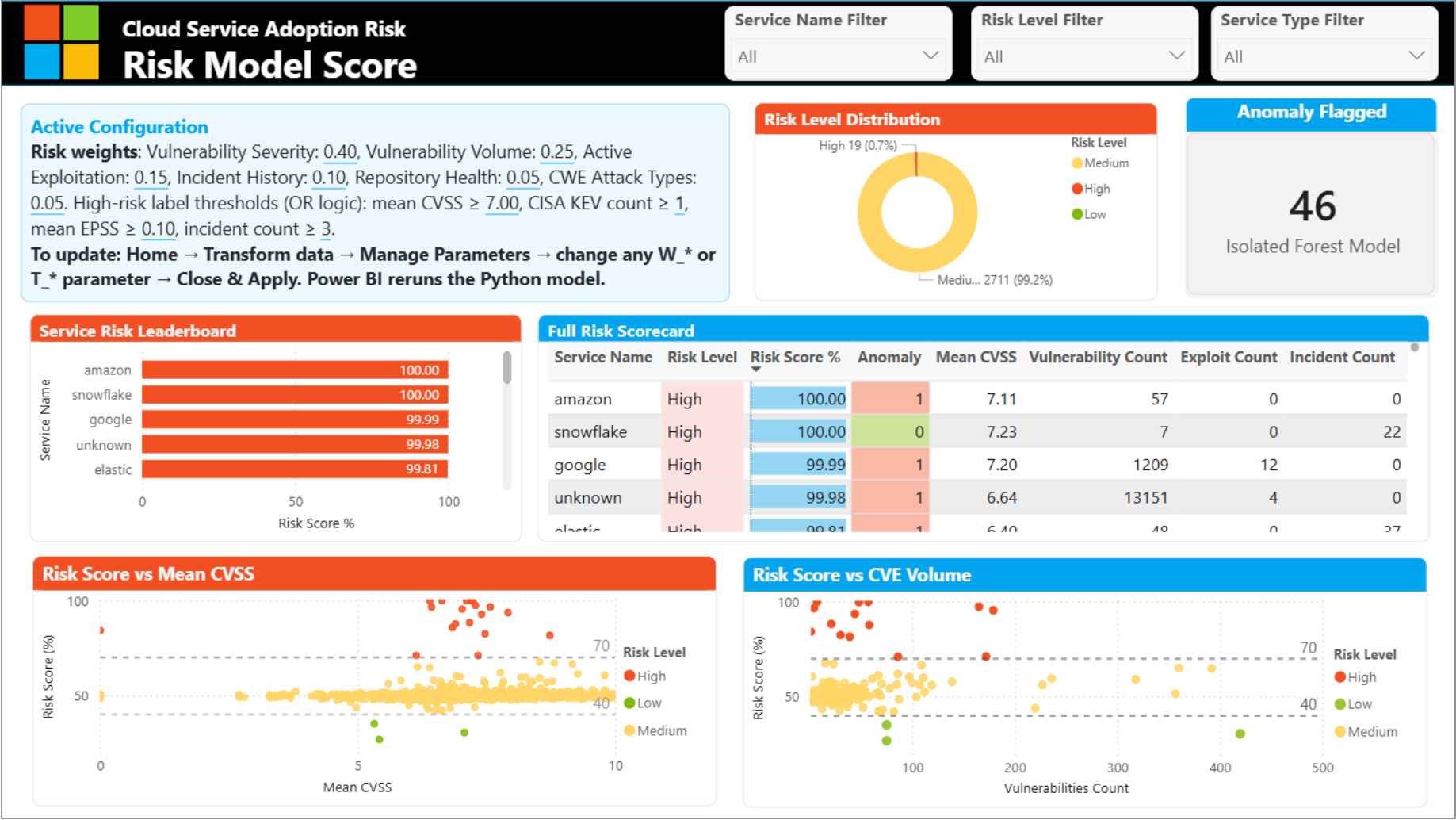
 Medium · 0.40 – 0.70

 High · 0.70 – 1.00

Power BI dashboard — Data Overview



Power BI dashboard — Data Overview



Operational value

The model translates fragmented security signals into a single ranked queue that procurement, security operations, and engineering teams can act on consistently.



Automated triage

Every service receives a calibrated risk probability and an anomaly flag on each pipeline run, replacing manual scoring.



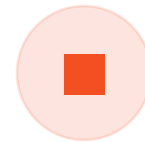
Configurable to org context

Each security team sets its own weights and thresholds — same pipeline serves different risk appetites without code changes.



Coverage for novel risk

Isolation Forest flags services that look low-risk on threshold metrics but combine attributes in statistically unusual ways.



Standardised vendor view

Risk scores exported to Power BI as a unified table — usable across procurement, SecOps, and engineering leadership.

Limitations & assumptions

The following items, drawn directly from the model documentation, frame how scores should be interpreted today and what is required to harden the pipeline further.

LIMITATIONS

No temporal modelling

All historical data is treated as equally recent. A service with one critical incident five years ago is weighted the same as one with a recent incident.

CWE encoding is dataset-relative

Only the top 20 most frequent CWEs are encoded. A rare but dangerous CWE (e.g. CWE-416 Use After Free) may be excluded if it does not appear frequently enough.

Anomaly scores are relative

Isolation Forest flags services unusual compared to the rest of the dataset. A dataset of uniformly high-risk services will still produce some low anomaly scores.

No live threat intelligence

KEV and EPSS data are only as current as the last database refresh — the model does not pull real-time feeds.

RECOMMENDED MITIGATIONS

Add temporal weighting

Apply exponential decay to incidents and CVEs so recent events carry more weight than older records.

Expand CWE coverage

Encode a curated list of high-severity CWEs in addition to the top-20 frequency cut, ensuring critical attack classes are never excluded.

Add population-stable scoring

Calibrate anomaly scores against a reference distribution so they remain interpretable across different dataset compositions.

Schedule live ingestion

Set up automated nightly pulls of KEV and EPSS so the pipeline always trains on current threat intelligence.

Recommended next steps

Five concrete actions to move the model pipeline from working prototype to production-grade tooling.

1

Schedule nightly pipeline runs

Write scores back to the risk_scores MySQL table for automated Power BI refresh.

2

Add SHAP analysis to XGBoost

Provide per-service explainability at scale, maintaining audit-grade transparency as model complexity grows.

3

Introduce time-decay weighting

Apply exponential decay to historical incidents and CVE severity so recent events count more.

4

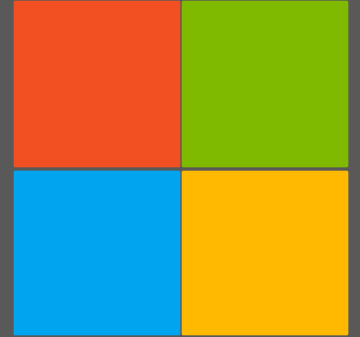
Expand training population

Add more services with incident history to balance the dataset and improve generalization.

5

Tune Isolation Forest contamination

Monitor false-positive rate over the first quarter of production use and adjust the contamination parameter accordingly.



Thank you

We welcome your questions.

TEAM

Arhum · Hansen · Kimmich · Nelson · Shelby · Shivani · Ximena

SPONSOR

Microsoft Corporation

DATE

May 2026